MTGNP — Magic: The Gathering Multiplayer Network Protocol

CSNETWK · S07 · T3 AY 2025–2026 · De La Salle University Manila

This is our implementation of the Magic: The Gathering Multiplayer Network Protocol (MTGNP) v1.0, as defined in RFC 0001. It is a TCP, message-oriented, client-server system for a two-player simplified game of Magic. The server is the only authority on the game state; the clients send actions and render whatever the server tells them.

The project is written in Python using the standard library only. No third-party package is used, so there is nothing to install and nothing to build. Every PDU is framed with a four (4) byte big-endian length prefix followed by a UTF-8 JSON payload, exactly as the RFC requires.

Requirements

- Python 3.9 or newer. We developed and tested on Python 3.14 for Windows, and the code also runs on macOS and Linux.
- No other software is required. There are no dependencies to install, no virtual environment to create, and no compilation step.

If `python` is not on your PATH on Windows, use the launcher `py` instead. Every command below works the same way with either one.

Build and Run Instructions

There is no build step. Clone or unzip the project, open a terminal in the folder that contains `server.py`, and run the files directly.

1. Start the server

```
python server.py --verbose
```

The server binds to port 4444 and waits in the LOBBY state. It accepts exactly two (2) clients and refuses any further connection with a `SERVER_FULL` error, as required by the RFC.

2. Start the first client

Open a second terminal in the same folder:

```
python client.py --player-id player_1 --deck decks/burn.txt --verbose
```

3. Start the second client

Open a third terminal:

```
python client.py --player-id player_2 --deck decks/control.txt --verbose
```

Once both clients have sent `PLAYER_READY`, the server shuffles both decks, sets both life totals to twenty (20), deals seven (7) cards each, flips a coin for the first player, and the mulligan step begins. From there the game is played entirely through the client prompts.

The two players must use different deck files. Both decks are drawn from one shared fixed card set and the RFC identifies permanents by card instance ID, so a card instance may only belong to one player. All five (5) sample decks in `decks/` use different cards, so any two of them can be paired. A deck that overlaps with the opponent's deck is rejected with `ILLEGAL_DECK`.

Server options

Option	Meaning
`--host <address>`	Interface to bind. Defaults to all interfaces.
`--port <number>`	Listening port. Defaults to 4444.
`--verbose`, `-v`	Start with verbose PDU logging turned on.
`--pretty`	Indent the JSON in verbose output across several lines.
`--time-limit-ms <ms>`	Priority timeout in milliseconds. Defaults to 300000.

Client options

Option	Meaning
`--player-id <name>`	The name claimed in `PLAYER_READY`. Required.
`--deck <path>`	Path to a deck file, for example `decks/burn.txt`. Required.
`--host <address>`	Server address. Defaults to 127.0.0.1.
`--port <number>`	Server port. Defaults to 4444.
`--verbose`, `-v`	Start with verbose PDU logging turned on.
`--pretty`	Indent the JSON in verbose output across several lines.

Run either program with `--help` to see the same list on the console.

Verbose Mode

Verbose mode prints every PDU that is sent and every PDU that is received, on both the server side and the client side, with a clear label for the direction and the PDU type. It can be turned on at startup and it can also be toggled while the program is running.

To enable it at startup, pass `--verbose` (or `-v`) to the server and to each client:

```
python server.py --verbose
python client.py --player-id player_1 --deck decks/burn.txt --verbose
python client.py --player-id player_2 --deck decks/control.txt --verbose
```

To toggle it at runtime:

- On the **server**, type `v` and press Enter. The server replies with the new state, either `verbose on` or `verbose off`.
- On a **client**, type `verbose` at any prompt and press Enter.

Adding `--pretty` prints the JSON body indented over several lines instead of on a single line. This is easier to read when demonstrating a large `GAME_STATE_UPDATE`, but it produces much longer output.

Playing a Game

The client prints the board after every update and prompts when it is your turn to act. Type `help` at any prompt for this same list.

Command	What it does
`pass`	Pass priority.
`cast <card_id> [target]`	Cast a spell. The mana payment is computed for you.
`land <card_id>`	Play a land. Main Phase only, once per turn.
`ability <permanent_id> [index] [target]`	Activate an ability of a permanent.
`concede`	Concede the game.
`keep [card_id ...]` / `mull`	Mulligan decision.
`discard <card_id> ...`	Discard down to seven (7) cards at Cleanup.
`attack [creature_id ...]`	Declare attackers. No IDs means no attack.
`block [blocker_id:attacker_id ...]`	Declare blockers. No pairs means no blocks.
`order <attacker_id> <blocker_id> ...`	Assign the combat damage order.
`yes [target]` / `no`	Accept or decline an optional trigger.
`state`	Reprint the board.
`hand`	List the cards in your hand.
`verbose`	Toggle PDU logging.
`help`	Print the command list.

A deck file lists one card per line, either as `<count> <card base>` (for example `4 lightning_bolt`, which expands to `lightning_bolt_001` through `lightning_bolt_004`) or as a single card instance ID such as `mountain_007`. A `#` starts a comment. See any file in `decks/` for a working example.

Running the Tests

```
python -m unittest discover -s tests
```

This runs ninety-one (91) tests in about eleven (11) seconds. They are split into two files. `tests/test_mtgnp.py` starts the real server on a real socket and talks to it with hand-written PDUs, which checks framing, sequence numbers, error codes, and the full lifecycle from LOBBY to GAME_OVER. `tests/test_rules.py` tests the rules engine directly with no sockets involved, which covers priority, the stack, combat, mana payment, and the card effects.

Project Structure

Each file matches one row of the Work Distribution Matrix, so the work of each member is easy to locate.

File	Responsibility
`server.py`, `client.py`	Entry points.
`mtgnp/protocol.py`	Framing, all twenty-five (25) PDU names, error codes, phase names.
`mtgnp/verbose.py`	Verbose logging and its toggle.
`mtgnp/cards.py`	The fixed card catalog, keywords, and mana sources.
`mtgnp/state.py`	Game state, and the per-player filtering that hides information.
`mtgnp/mana.py`	Validates a declared mana payment and taps the sources.
`mtgnp/effects.py`	Spell effects, activated abilities, triggers, target legality.
`mtgnp/engine.py`	Sequence numbers, sending, waiting for actions, state-based actions.
`mtgnp/priority.py`	Priority windows, the LIFO stack, resolution and fizzling.
`mtgnp/combat.py`	The combat sub-state machine and damage assignment.
`mtgnp/turns.py`	Every phase and step, in order.
`mtgnp/lifecycle.py`	LOBBY, GAME_SETUP, MULLIGAN, GAME_OVER, and session restart.
`mtgnp/server.py`	TCP sockets, accepting exactly two (2) clients, dispatch, PONG.
`mtgnp/client.py`	Rendering, prompts, the heartbeat, and deck loading.
`data/`	The master card list, loaded at runtime.
`decks/`	Five (5) sample decks.
`tests/`	The test suites described above.

The server uses one reader thread for each client socket, and those threads only push received PDUs onto a queue. A single game thread owns all of the rules, so no rules code needs a lock. `PING` is answered directly in the reader thread because it does not touch the game state.

Work Distribution Matrix

Member	Git author
Member 1	festivities `<admin@festivity.moe>`
Member 2	Marco Gerard D. Mendoza `<marco_gerard_mendoza@dlsu.edu.ph>`
Member 3	awynee `<164032813+awynee@users.noreply.github.com>`

P marks the primary author, who designed and wrote the component. **S** marks a supporting member, who reviewed it, tested it, or contributed fixes.

Task / Feature	Member 1	Member 2	Member 3
TCP Server: connection handling, framing, dispatch	**P**	S	
Game lifecycle: LOBBY, GAME_SETUP, MULLIGAN logic	S	**P**	
Turn & phase engine (all phases/steps, transitions)		S	**P**
Priority & Stack logic, spell/ability resolution	S		**P**
Combat system (attackers, blockers, damage)		S	**P**
Client implementation & state rendering	S		**P**
PDU serialisation/deserialisation (all 25 PDU types)	**P**	S	
Error handling, PING/PONG heartbeat, disconnect logic	**P**		S
Verbose mode (client + server PDU logging, toggle on/off)	**P**		S
Card catalog, mana payment & card effects		**P**	S
Game state management & hidden-information filtering		**P**	S
Testing & interoperability	**P**	S	S
README / documentation / AI disclosure	S	**P**	S

The two bonus criteria, implementing every card ability and adding extra features such as a graphical interface or a spectator client, were not attempted. This submission targets the one hundred (100) point base rubric.

AI Usage

We used AI tools while working on this machine problem. They are listed below with a description of how each one was used.

Tool	How it was used
Claude (Anthropic), through the Claude Code command line tool	Drafting and reviewing large parts of the Python source, explaining sections of the RFC that contradict themselves, generating test cases, and writing this README.

Tool	How it was used
OpenCode, an open-source terminal coding agent	Used in an earlier attempt at the same project, mainly for scaffolding the module layout and the first version of the PDU framing code.

How we worked with these tools:

- We wrote the prompts around the RFC itself. Where the RFC was ambiguous we decided on the reading first, then asked the tool to implement that reading, rather than letting the tool choose for us.
- Every piece of generated code was read line by line, edited to match the rest of the project, and tested. The ninety-one (91) tests in `tests/` are the main way we checked the output, and we also played full games over real sockets.
- The deviations listed in the next section are our own decisions. Each one was traced back to a specific section of the RFC before it was accepted.
- No AI output was shared with or taken from another group, and no session from another student was reused.

Every member can explain any part of the submission, including the parts that were drafted with AI assistance.

Known Limitations and Deviations from the RFC

Where the RFC contradicts itself

Section 5.3 designates Section 10.2 as authoritative for field names, so wherever the prose examples disagree with Section 10.2 we follow Section 10.2. Our client accepts both spellings so that it can still talk to another group's server.

Field	What we send, per §10.2	What the prose examples show
Hand	`{"player_1": [...]}` object	A bare array
Land flag	`land_played_this_turn`	`land_played`
Creature flag	`summoning_sick`	`summoning_sickness`
State change kind	`change_type`	`type`

Deliberate protocol decisions

- **Sequence numbers** are a plain counter that increases by one (1) for every PDU the server sends, and it keeps counting across games in the same session. Section 5.4 allows this. A broadcast to both players shares one sequence number, while each personalised `GAME_STATE_UPDATE` gets its own.
- **After an invalid action, the re-issued `PRIORITY_GRANT` carries a new sequence number** and that new number becomes the valid token. This matches the worked example in Section 5.4. The wording in Section 11 says the same number is reused, which contradicts the example, so we followed the example.
- **The declaration steps have no request PDU.** `DECLARE_ATTACKERS`, `DECLARE_BLOCKERS` and `ASSIGN_DAMAGE_ORDER` echo the sequence number of the `PHASE_TRANSITION`, as shown in Section 5.4. No `PRIORITY_GRANT` comes before them.

- **Every accepted action is followed by a state update.** The RFC does not require this, but `STACK_PUSH` says nothing about zones, so without the extra update a client cannot tell that a card has left a hand.
- **The visible state carries one extra key,** `combat`, holding the attackers, the blocks, and the damage order. It is not named in Section 10.2, but it is public information, and the attacking player otherwise has no way to learn how they were blocked, which they need in order to send `ASSIGN_DAMAGE_ORDER`. Nothing is renamed, so a stricter client can simply ignore the key.
- **The two deck lists must be disjoint**, for the reason given earlier. Deck validation also rejects a card whose copy count is higher than the "Copies in Set" column of the master card list. Both cases return `ILLEGAL_DECK`.
- **Summoning sickness is cleared at the controller's Untap Step**, following Section 3, and not at Cleanup as one of the examples shows.
- **The priority timeout defaults to 300000 ms** instead of the 60000 ms used in the examples, so that a live demo is not timed out while a player is thinking. It can be set with `--time-limit-ms`.
- **A disconnect or a priority timeout ends the game** with `GAME_OVER(DISCONNECT)`. The surviving player keeps their connection, the server returns to LOBBY, and the empty slot is opened for a replacement, which is what Section 4.2 prescribes for timeouts. Rejoining a game that is already in progress is not supported.
- **The client folds the phase, turn number and active player in from** `PHASE_TRANSITION`, because the server does not follow every transition with a state update.

Card effects

The rubric asks for at least five (5) card effects. We implemented seventeen (17) spell effects, four (4) activated abilities (Prodigal Sorcerer, Royal Assassin, Rod of Ruin, and Millstone), and three (3) triggered abilities (Gray Merchant, Gravedigger, and Goblin Guide). Mana abilities, meaning the basic lands, Llanowar Elves, Elvish Mystic and Sol Ring, are handled implicitly as Section 7.5 allows. The keywords enforced are haste, first strike, double strike, defender, flying and vigilance.

The following are **not** implemented. Casting one of these is rejected with `ILLEGAL_ACTION` rather than silently doing nothing:

- Keywords and mechanics: prowess, protection, hexproof, kicker, madness, suspend, regenerate, illusion, and auras such as Pacifism. Trample is excluded by Section 9.7 of the RFC itself.
- Spells: Skullcrack, Rift Bolt, Mana Leak, Vines of Vastwood, Naturalize, Dark Ritual, Ponder, Rampant Growth, Mind Rot, and the land search half of Path to Exile. Healing Salve applies its first mode only.

The common reason for these is that they need a choice from a player in the middle of resolving a spell, and the RFC defines no PDU for asking that question. We avoided the ones that would have required inventing a message the protocol does not have. Implementing all of them is bonus work, which this submission does not attempt.

Rebuilding This Document

`README.pdf` is generated from `README.md`, so the source of truth is the Markdown file. After editing it, run:

```
build_readme.bat
```

The script converts `README.md` to HTML with `tools/md2pdf.py`, which uses the Python standard library only, then prints that HTML to `README.pdf` with a headless Microsoft Edge or Google Chrome, whichever it finds first. On macOS or Linux, or if you prefer not to use the batch file, run the same step directly:

```
python tools/md2pdf.py README.md README.pdf
```

Neither script is part of the protocol implementation. They exist only to keep the submitted PDF in step with the Markdown source.